Module 2 — Thursday Stretch: Experiment Tracker

Type: Stretch Assignment — optional

Prerequisite: Complete Integration 2 and Integration Challenge Tier 1 (train/test split + evaluation metrics). **If you haven't done Challenge Tier 1, go back and complete it first** — the stretch builds on those skills.

Submission: In your integration repo, create a branch `stretch-experiment-tracker`, add your files, open a PR to `main`, and paste the PR URL into TalentLMS → Module 2 → Thursday Stretch

---

Objective

Build a CLI tool that runs your housing price model multiple times with different configurations, logs results, and produces a ranked leaderboard. This is batch experimentation infrastructure — the kind of tooling ML engineers build to systematically explore which settings produce the best model.

What You'll Learn

1. **Systematic experimentation** — not just trying random things, but exhaustively searching a defined space
2. **Experiment tracking** — logging configs and results so experiments are reproducible and comparable
3. **Parameterized training** — separating model/training configuration from execution logic

These are the same patterns used by tools like Weights & Biases, MLflow, and Optuna — you are building a minimal version from scratch.

Quick Reference

You used these concepts in Integration Challenge Tier 1. Here's a quick reference for the stretch.

Train/Test Split

In the base integration, you trained and evaluated on the same data. This is a problem — a model can memorize training data and appear accurate without actually learning generalizable patterns. To measure real performance, you split your data:

- **Training set (80%):** the model learns from this data
- **Test set (20%):** held out, never seen during training — used only to measure how well the model performs on new data

With tensors, the split is straightforward:

```
# Shuffle indices (use a fixed seed so every experiment uses the same split)
torch.manual_seed(42)
indices = torch.randperm(len(X_tensor))
X_shuffled = X_tensor[indices]
y_shuffled = y_tensor[indices]

# Split at 80%
split = int(0.8 * len(X_tensor))
X_train, X_test = X_shuffled[:split], X_shuffled[split:]
y_train, y_test = y_shuffled[:split], y_shuffled[split:]
```

Train the model on `X_train` / `y_train`. Score it on `X_test` / `y_test`. The test set measures how the model performs on data it has never seen.

MAE (Mean Absolute Error)

MAE measures the average size of prediction errors in the original units (JOD). Lower is better.

```
# After training, generate test predictions
with torch.no_grad():
    test_preds = model(X_test).numpy().flatten()
test_actual = y_test.numpy().flatten()

mae = np.mean(np.abs(test_actual - test_preds))
```

If your MAE is 8,000 JOD, it means your predictions are off by about 8,000 JOD on average. For apartments priced 30,000–150,000 JOD, that gives you a sense of how useful the model is.

R² (Coefficient of Determination)

R² measures how much of the variation in prices your model explains. It ranges from 0 to 1 (higher is better). An R² of 0.85 means the model explains 85% of the variation in apartment prices.

```
ss_res = np.sum((test_actual - test_preds) ** 2)
ss_tot = np.sum((test_actual - np.mean(test_actual)) ** 2)
r_squared = 1 - (ss_res / ss_tot)
```

An R² of 0.0 means the model is no better than predicting the average price for every apartment. An R² above 0.8 is strong for a simple model like this.

What to Build

Create an `experiment_tracker.py` script that:

1. Designs a hyperparameter search

Your integration task used fixed values: `learning_rate=0.01`, `hidden_size=32`, `num_epochs=100`. But how do you know those are good choices? You don't — until you try alternatives.

Design your own search grid. Choose at least 3 hyperparameters to vary, and select a range of values for each. Your grid should produce at least 30 configurations.

To help you choose reasonable ranges:

- **Learning rate** controls how much the model adjusts its weights on each step. Too high and the model overshoots (loss jumps around or increases). Too low and training is slow (loss barely moves). Typical values for Adam: somewhere between 0.0001 and 0.1. Try values across this range.
- **Hidden size** is the number of neurons in the hidden layer. More neurons = more capacity to learn complex patterns, but also more risk of memorizing noise. Your base model used 32. Try smaller and larger values.
- **Number of epochs** is how many times the model sees the full training data. More epochs gives the model more chances to learn, but eventually the model starts memorizing the training set instead of learning general patterns. Your base model used 100. Try fewer and more.

You may also experiment with other hyperparameters if you wish — optimizer choice (`Adam` vs `SGD`), number of hidden layers, or activation functions.

2. Runs each configuration

For each combination in your grid:

- Build a `HousingModel` with the specified `hidden_size` (modify the constructor to accept this as a parameter)
- Train with the specified `learning_rate` and `num_epochs`
- Use an 80/20 train/test split (same split for every run — fix the random seed)
- Record: config, final train loss, final test loss, test MAE, test R², training time in seconds

3. Targets a test MAE below 10,000 JOD

Your goal is to find a configuration where the test MAE drops below **10,000 JOD**. This means your model's predictions are off by less than 10,000 JOD on average — useful accuracy for apartment pricing.

If your best configuration doesn't reach this target, that's OK — document:

- Your best MAE and which configuration produced it
- What patterns you see in your results (do more epochs help? does a larger hidden size help?)
- What you would try next to push the MAE lower

The learning is in the analysis, not just the number.

4. Logs results

Save all runs to `experiments.json` — a list of experiment records, each containing the config and all metrics. This file is your experiment database.

5. Prints a leaderboard

After all runs complete, print a ranked table of the top 10 configurations sorted by test MAE (lowest first):

```
Rank | LR      | Hidden | Epochs | Test MAE    | Test R²   | Time (s)
-----|---------|--------|--------|-------------|-----------|--------
  1  | ...     |   ...  |  ...   | ...         | ...       | ...
  2  | ...     |   ...  |  ...   | ...         | ...       | ...
  ...
```

6. Produces a summary visualization

Save a chart (`experiment_summary.png`) that helps you see which regions of your hyperparameter space perform best. For example: test MAE vs. learning rate, with separate lines or markers for each hidden size.

Starting Point

You already have a working `train.py` from the integration task. The main changes:

1. **Modify `HousingModel.__init__`** to accept `hidden_size` as a parameter instead of hardcoding 32
2. **Extract the training logic** into a function that accepts config parameters and returns metrics
3. **Add train/test split** using the pattern shown in the "Before You Start" section above
4. **Build the experiment loop** that iterates over all combinations in your grid

---

Design Decisions

These are yours to make:

- **Which hyperparameters to search and what ranges to use** — justify your choices based on what you observe
- **How to structure the JSON output** — flat list of records? Nested by hyperparameter? Your choice.
- **How to handle the experiment runs** — depending on your grid size, this may take a few minutes. Consider printing a progress indicator.
- **Whether to add early stopping** — optional but useful for long runs. If you completed Integration Challenge 3, you already have a framework for this.

Submit

1. In your integration repo, create a branch `stretch-experiment-tracker`
2. Add `experiment_tracker.py`, `experiments.json`, and `experiment_summary.png`
3. Open a PR to `main`
4. Paste your PR URL into TalentLMS → Module 2 → Thursday Stretch

---

Module 2 of 12 — Applied AI & ML Systems